



DESENVOLVIMENTO WEB E MOBILE

DESENVOLVIMENTO WEB E MOBILE - ANDROID - PERSISTÊNCIA DE DADOS NO ANDROID

Prof. Me. Gustavo Ferreira Palma

PAUTA

- Persistência de Dados no Android
- Persistência de utilizando Bancos de Dados Locais
 - SQLite
 - Android ROOM e Coroutines
 - Integrando bibliotecas ao projeto Android
- Exmeplo Prático

PERSISTÊNCIA DE DADOS NO ANDROID

PERSISTÊNCIA DE DADOS NO ANDROID

- A persistência de dados consiste, basicamente, no salvamento de dados. Persistir dados significa salvá-los e no Android isso pode ser feito de diversas formas, tendo como principais:
 - Arquivos: Podem ser arquivos binários ou arquivos de texto, são salvos na memória *Flash* do telefone, podem ser públicos ou privados
 - DataStore(SharedPreferences): O DataStore, é a versão moderna do SharedPreferences, que é uma forma rápida para armazenar preferências do usuário, ou mesmo dados com baixo critério de confidencialidade
 - Bancos de Dados: Todo dispositivo Android possui embarcado uma versão do Banco de Dados SQLite, um pequeno banco de dados relacional, que permite a persistência segura de dados, suportando operações complexas, como JOINS ou criação de VIEWS

PERSISTÊNCIA DE DADOS NO ANDROID

- Apesar do SQLite suportar tabelas com até 1GB de tamanho, e lidar bem com operações complexas, originalmente as aplicações Android não foram pensadas para demandar um grande armazenamento de dados local.
- Em outras palavras, podemos criar aplicações que consigam gerenciar grandes volumes de dados e o banco de dados pode lidar com eles, contudo não devemos.
- Isso se deve em grande parte, por uma série de limitações e possibilidade de perda para os dados, como por exemplo:
 - Se os dados forem armazenados apenas localmente, o usuário pode por descuido limpar o armazenamento local do telefone simplesmente excluir todos os dados sem possibilidade de recuperação.
 - Caso o usuário migre para um telefone mais novo, os dados não serão transferidos.
- Para evitar esses problemas, o desenvolvedor deve optar pela integração da sua aplicação com serviços de nuvem. Esse conteúdo será abordado posteriormente

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - SQLITE

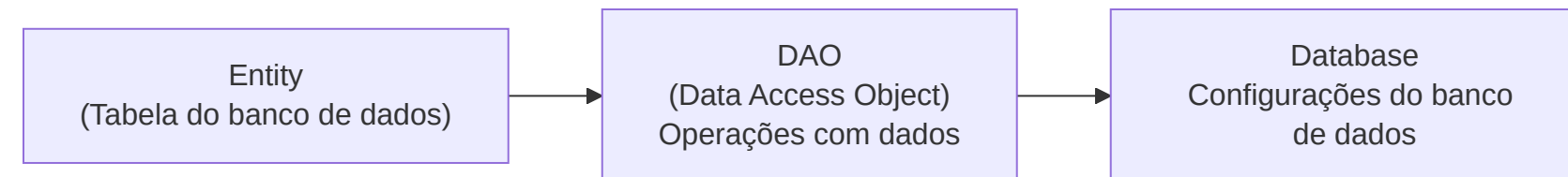
- O SQLite é um SGBD (Sistema Gerenciador de Banco de Dados) relacional, projetado para ambientes embarcados;
- Ao contrário de SGBDs como o MySQL ou SQL Server, o SQLite não precisa de um servidor rodando, ele implementa a estratégia 'Self Contained File'
- É um SGBD multiplataforma de código aberto

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- O Android Room, é um pequeno ORM (Object-Relational Mapping) desenvolvido pela Google para Android
- O uso do Room, é encorajado pois reduz a complexidade e a necessidade de escrever códigos repetitivos uma vez que ele pode automatizar a geração de códigos SQL e mesmo alguns códigos de manipulação de dados

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- Hierarquia entre componentes ROOM



PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- Fluxo de Operações com Android Room



PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

```
@Entity
data class Pessoa(
    @PrimaryKey(autoGenerate = true)
    val id: Int,
    val nome: String,
    val idade: Int
)
```

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

```
@Dao
interface PessoaDAO {

    @Insert
    suspend fun inserir(pessoa: Pessoa)

    @Delete
    suspend fun excluir(pessoa: Pessoa)

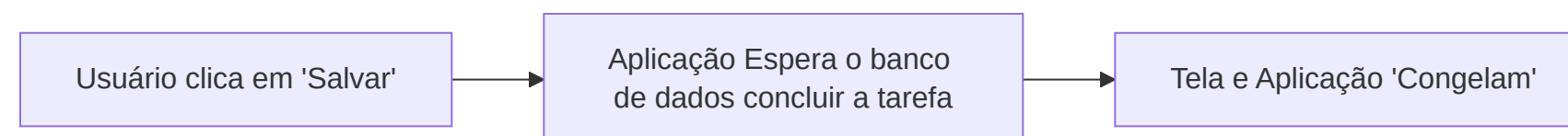
    @Query("SELECT * FROM Pessoa")
    suspend fun listar(): List<Pessoa>
}
```

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

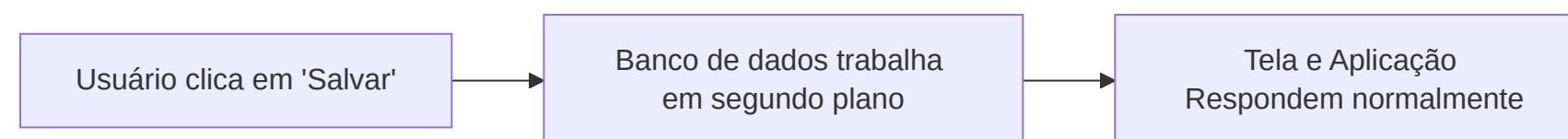
```
@Database(  
    entities = [Pessoa::class],  
    version = 1  
)  
abstract class AppDatabase : RoomDatabase() {  
    abstract fun pessoaDAO(): PessoaDAO  
}
```


PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- Compreendendo a ideia de funcionamento do Room, precisamos entender o porque devemos usar Coroutines, e qual a ideia por trás de sua aplicação
- Primeiro, devemos entender como os processos funcionam sem as Coroutines:



- Com o uso de Coroutines:



PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- Quando estamos desenvolvendo métodos e funções utilizando os recursos das Coroutines, uma palavra chave muda totalmente o fluxo de execução da aplicação

```
suspend fun salvarPessoa() {  
    dao.insert(pessoa)  
}
```

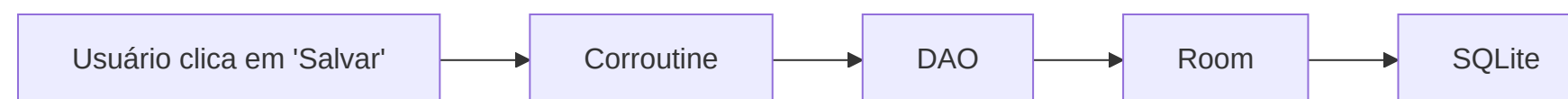
- Uma função marcada com suspend pode ser pausada e retomada sem bloquear a thread;
- Ela só pode ser chamada dentro de outra coroutine ou de uma função suspend.

PERSISTÊNCIA DE UTILIZANDO BANCOS DE DADOS LOCAIS - ANDROID ROOM E COUROUTINES

- A gravação acontece em uma thread apropriada para operações de entrada e saída (I/O), mantendo a interface responsiva.

```
lifecycleScope.launch() {  
    dao.insert(pessoa)  
}
```

• Fluxo Completo



INTEGRANDO BIBLIOTECAS AO PROJETO ANDROID

INTEGRANDO BIBLIOTECAS AO PROJETO ANDROID

- Mesmo o Room e as Coroutines sendo bibliotecas padrão do kotlin é necessário declarar o uso nos arquivos gradle

```
plugins {  
    ...  
    alias(libs.plugins.ksp)  
}  
...  
dependencies {  
    ...  
    implementation(libs.androidx.room.runtime)  
    implementation(libs.androidx.room.ktx)  
    ksp(libs.androidx.room.compiler)  
    ...  
}
```

EXEMPLO PRÁTICO

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br